Lab#07
Sorting Algorithms II

7.1 Merge Sort

Merge sort is defined as a *sorting algorithm* that works by dividing an array into smaller subarrays, sorting each subarray, and then merging the sorted subarrays back together to form the final sorted array.

In simple terms, we can say that the process of merge sort is to divide the array into two halves, sort each half, and then merge the sorted halves back together. This process is repeated until the entire array is sorted.

7.1.1 How does Merge Sort work?

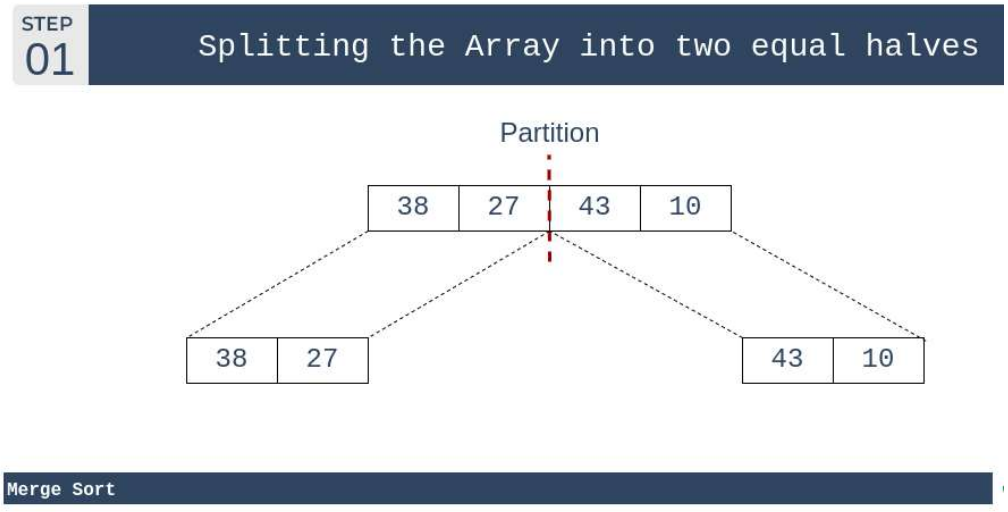
Merge sort is a recursive algorithm that continuously splits the array in half until it cannot be further divided i.e., the array has only one element left (an array with one element is always sorted). Then the sorted subarrays are merged into one sorted array.

See the below illustration to understand the working of merge sort.

7.1.2 Illustration:

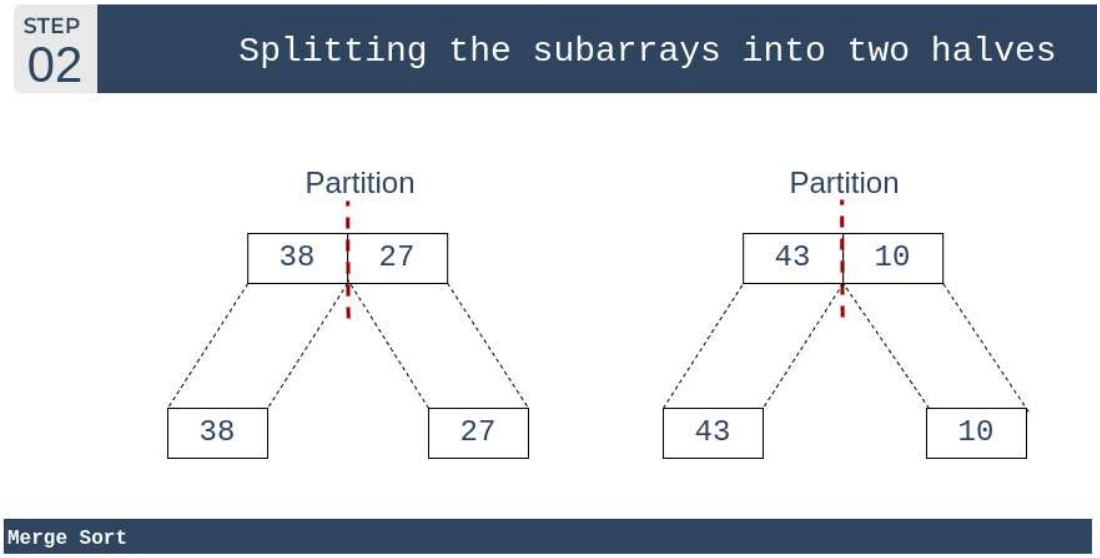
Lets consider an array ***arr[] = {38, 27, 43, 10}***

Initially divide the array into two equal halves:



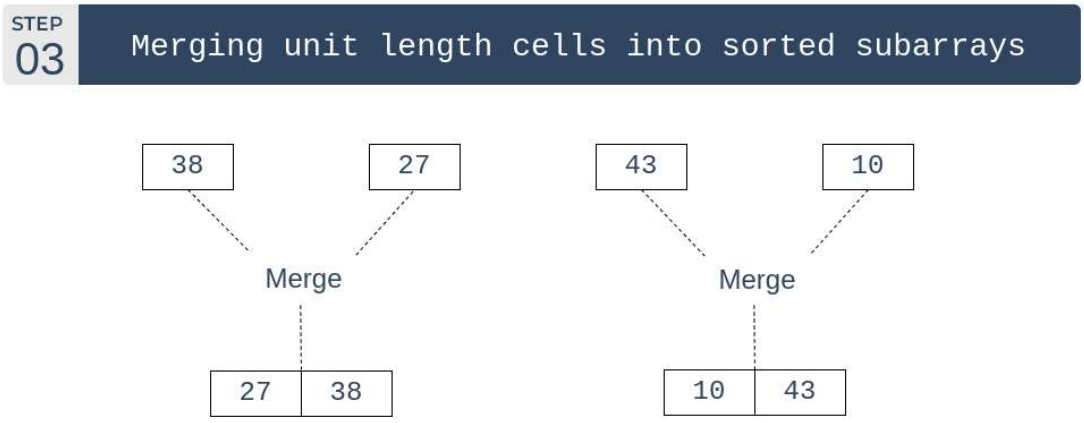
Merge Sort: Divide the array into two halves

- These subarrays are further divided into two halves. Now they become array of unit length that can no longer be divided and array of unit length are always sorted.



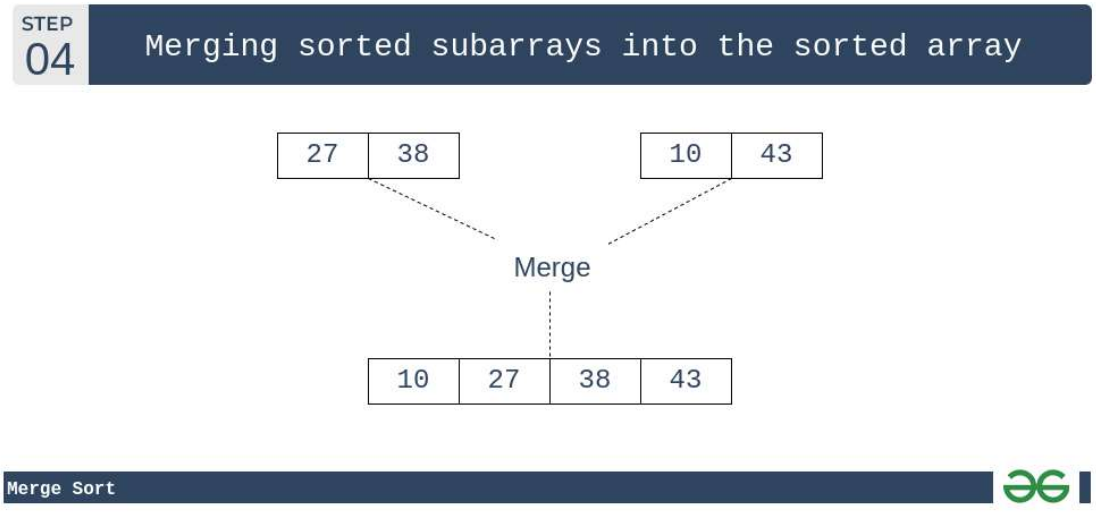
Merge Sort: Divide the subarrays into two halves (unit length subarrays here)

These sorted subarrays are merged together, and we get bigger sorted subarrays.



Merge Sort: Merge the unit length subarrys into sorted subarrays

This merging process is continued until the sorted array is built from the smaller subarrays.



Merge Sort: Merge the sorted subarrys to get the sorted array

The following diagram shows the complete merge sort process for an example array {38, 27, 43, 3, 9, 82, 10}.

7.1.3 Merge Sort Implementation in C++

Below is the Code implementation of Merge Sort.

```
// C++ program for Merge Sort
#include <bits/stdc++.h>
using namespace std;

// Merges two subarrays of array[].
// First subarray is arr[begin..mid]
// Second subarray is arr[mid+1..end]
void merge(int array[], int const left, int const mid,
           int const right)
{
    int const subArrayOne = mid - left + 1;
    int const subArrayTwo = right - mid;
```

```
// Create temp arrays
auto *leftArray = new int[subArrayOne],
     *rightArray = new int[subArrayTwo];

// Copy data to temp arrays leftArray[] and rightArray[]
for (auto i = 0; i < subArrayOne; i++)
    leftArray[i] = array[left + i];
for (auto j = 0; j < subArrayTwo; j++)
    rightArray[j] = array[mid + 1 + j];

auto indexOfSubArrayOne = 0, indexOfSubArrayTwo = 0;
int indexOfMergedArray = left;

// Merge the temp arrays back into array[left..right]
while (indexOfSubArrayOne < subArrayOne
       && indexOfSubArrayTwo < subArrayTwo) {
    if (leftArray[indexOfSubArrayOne]
        <= rightArray[indexOfSubArrayTwo]) {
        array[indexOfMergedArray]
            = leftArray[indexOfSubArrayOne];
        indexOfSubArrayOne++;
    }
    else {
        array[indexOfMergedArray]
            = rightArray[indexOfSubArrayTwo];
        indexOfSubArrayTwo++;
    }
    indexOfMergedArray++;
}
```

```
    // Copy the remaining elements of
    // left[], if there are any
    while (indexOfSubArrayOne < subArrayOne) {
        array[indexOfMergedArray]
            = leftArray[indexOfSubArrayOne];
        indexOfSubArrayOne++;
        indexOfMergedArray++;
    }

    // Copy the remaining elements of
    // right[], if there are any
    while (indexOfSubArrayTwo < subArrayTwo) {
        array[indexOfMergedArray]
            = rightArray[indexOfSubArrayTwo];
        indexOfSubArrayTwo++;
        indexOfMergedArray++;
    }
    delete[] leftArray;
    delete[] rightArray;
}

// begin is for left index and end is right index
// of the sub-array of arr to be sorted
void mergeSort(int array[], int const begin, int const end)
{
    if (begin >= end)
        return;

    int mid = begin + (end - begin) / 2;
    mergeSort(array, begin, mid);
```

```
    mergeSort(array, mid + 1, end);
    merge(array, begin, mid, end);
}

// UTILITY FUNCTIONS
// Function to print an array
void printArray(int A[], int size)
{
    for (int i = 0; i < size; i++)
        cout << A[i] << " ";
    cout << endl;
}

// Driver code
int main()
{
    int arr[] = { 12, 11, 13, 5, 6, 7 };
    int arr_size = sizeof(arr) / sizeof(arr[0]);

    cout << "Given array is \n";
    printArray(arr, arr_size);

    mergeSort(arr, 0, arr_size - 1);

    cout << "\nSorted array is \n";
    printArray(arr, arr_size);
    return 0;
}

// This code is contributed by Mayank Tyagi
```

```
// This code was revised by Joshua Estes
```

Output

```
Given array is
12 11 13 5 6 7
Sorted array is
5 6 7 11 12 13
```

7.1.4 Complexity Analysis of Merge Sort:

Time Complexity: O(N log(N)), Merge Sort is a recursive algorithm and time complexity can be expressed as following recurrence relation.

$T(n) = 2T(n/2) + \theta(n)$

The above recurrence can be solved either using the Recurrence Tree method or the Master method. It falls in case II of the Master Method and the solution of the recurrence is θ(Nlog(N)). The time complexity of Merge Sort isθ(Nlog(N)) in all 3 cases (worst, average, and best) as merge sort always divides the array into two halves and takes linear time to merge two halves.

Auxiliary Space: O(N), In merge sort all elements are copied into an auxiliary array. So N auxiliary space is required for merge sort.

7.1.5 Applications of Merge Sort:

- **Sorting large datasets:** Merge sort is particularly well-suited for sorting large datasets due to its guaranteed worst-case time complexity of O(n log n).
- **External sorting:** Merge sort is commonly used in external sorting, where the data to be sorted is too large to fit into memory.
- **Custom sorting:** Merge sort can be adapted to handle different input distributions, such as partially sorted, nearly sorted, or completely unsorted data.
- Inversion Count Problem

7.1.6 Advantages of Merge Sort:

Stability: Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.

Guaranteed worst-case performance: Merge sort has a worst-case time complexity of O(N logN), which means it performs well even on large datasets.

Parallelizable: Merge sort is a naturally parallelizable algorithm, which means it can be easily parallelized to take advantage of multiple processors or threads.

7.1.7 Drawbacks of Merge Sort:

- **Space complexity:** Merge sort requires additional memory to store the merged sub-arrays during the sorting process.
- **Not in-place:** Merge sort is not an in-place sorting algorithm, which means it requires additional memory to store the sorted data. This can be a disadvantage in applications where memory usage is a concern.
- **Not always optimal for small datasets:** For small datasets, Merge sort has a higher time complexity than some other sorting algorithms, such as insertion sort. This can result in slower performance for very small datasets.

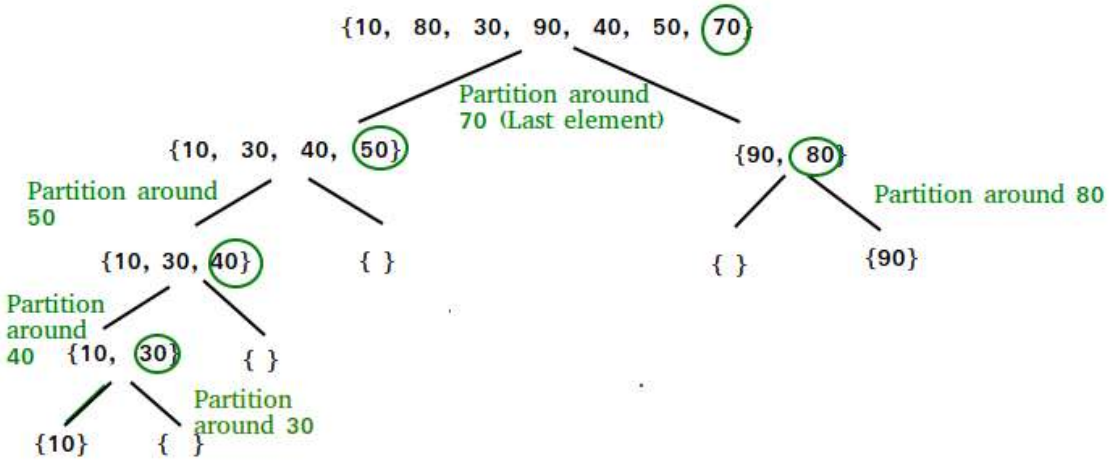
7.2 QuickSort – Data Structure and Algorithm C++

QuickSort is a sorting algorithm based on the Divide and Conquer algorithm that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.

7.2.1 How does QuickSort work?

The key process in ***quickSort*** is a ***partition()***. The target of partitions is to place the pivot (any element can be chosen to be a pivot) at its correct position in the sorted array and put all smaller elements to the left of the pivot, and all greater elements to the right of the pivot.

Partition is done recursively on each side of the pivot after the pivot is placed in its correct position and this finally sorts the array.



How Quicksort works

7.2.2 Choice of Pivot:

There are many different choices for picking pivots.

- Always pick the first element as a pivot.
- Always pick the last element as a pivot (implemented below)
- Pick a random element as a pivot.
- Pick the middle as the pivot.

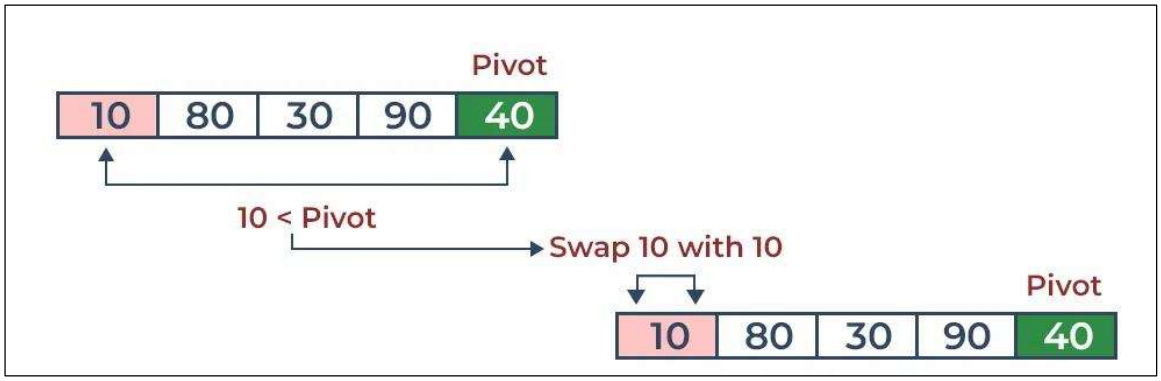
7.2.3 Partition Algorithm:

The logic is simple, we start from the leftmost element and keep track of the index of smaller (or equal) elements as ***i***. While traversing, if we find a smaller element, we swap the current element with arr[i]. Otherwise, we ignore the current element.

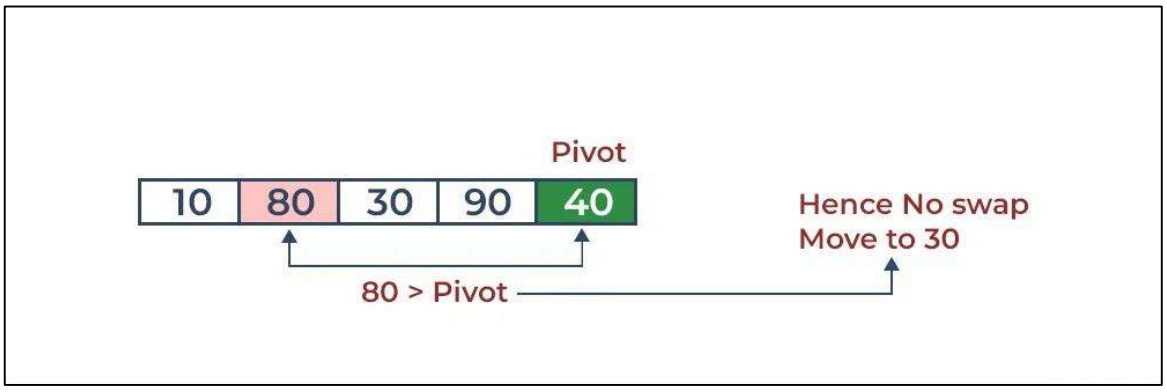
Let us understand the working of partition and the Quick Sort algorithm with the help of the following example:

Consider: arr[] = {10, 80, 30, 90, 40}.

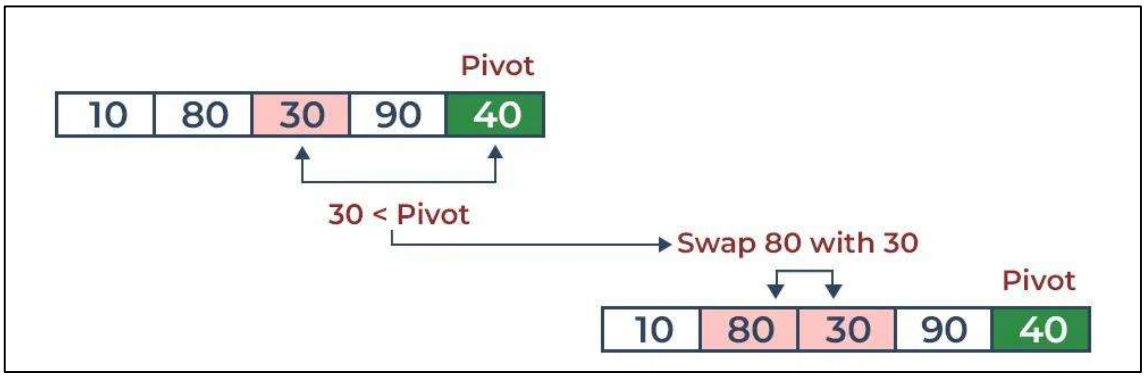
Compare 10 with the pivot and as it is less than pivot arrange it accordingly.



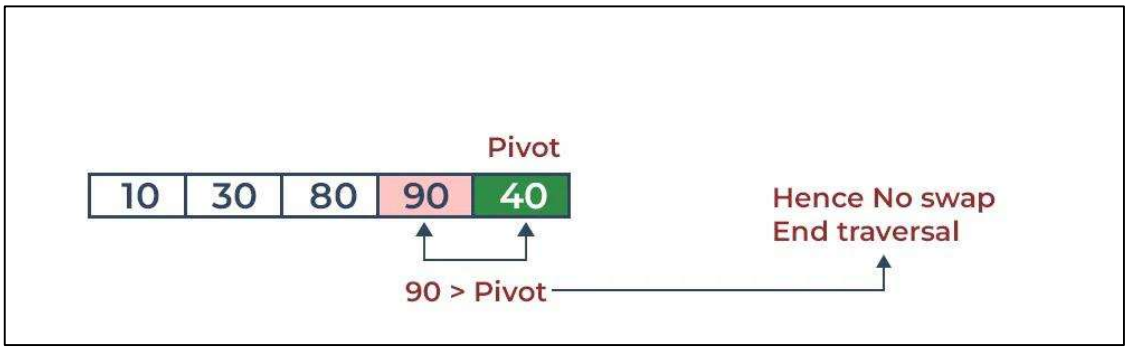
- Compare 80 with the pivot. It is greater than pivot.



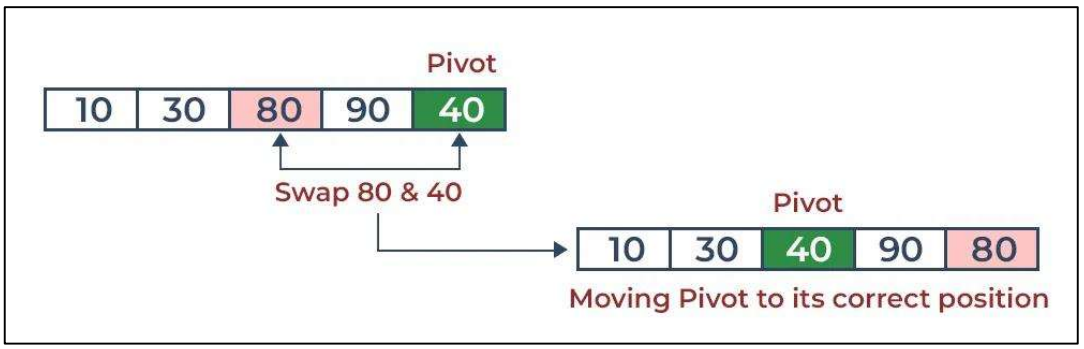
- Compare 30 with pivot. It is less than pivot so arrange it accordingly.



- Compare 90 with the pivot. It is greater than the pivot.



- Arrange the pivot in its correct position.

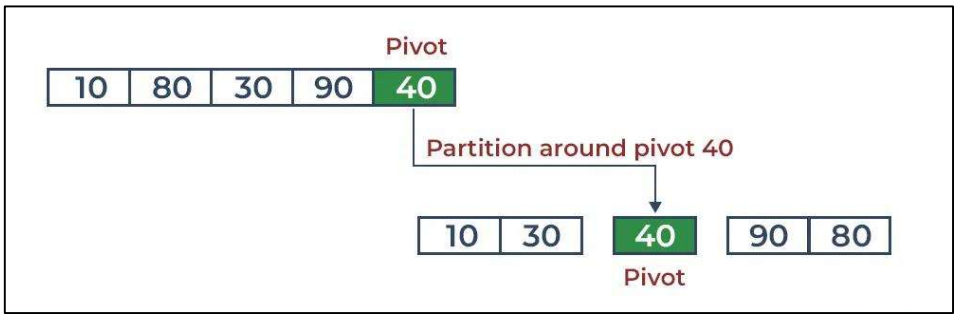


7.2.4 Illustration of Quicksort:

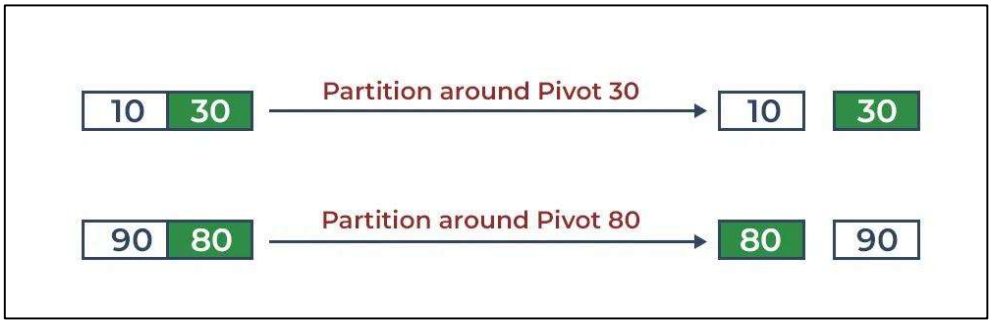
As the partition process is done recursively, it keeps on putting the pivot in its actual position in the sorted array. Repeatedly putting pivots in their actual position makes the array sorted.

Follow the below images to understand how the recursive implementation of the partition algorithm helps to sort the array.

Initial partition on the main array:



Partitioning of the subarrays:



7.2.5 Code implementation of the Quick Sort

Below is the implementation of the Quicksort:

```
// C++ code to implement quicksort

#include <bits/stdc++.h>
using namespace std;

// This function takes last element as pivot,
// places the pivot element at its correct position
// in sorted array, and places all smaller to left
// of pivot and all greater elements to right of pivot
int partition(int arr[], int low, int high)
{
    // Choosing the pivot
    int pivot = arr[high];

    // Index of smaller element and indicates
    // the right position of pivot found so far
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {

        // If current element is smaller than the pivot
        if (arr[j] < pivot) {
```

```
            // Increment index of smaller element
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return (i + 1);
}

// The main function that implements QuickSort
// arr[] --> Array to be sorted,
// low --> Starting index,
// high --> Ending index
void quickSort(int arr[], int low, int high)
{
    if (low < high) {

        // pi is partitioning index, arr[p]
        // is now at right place
        int pi = partition(arr, low, high);

        // Separately sort elements before
        // partition and after partition
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Driver Code
int main()
{
    int arr[] = { 10, 7, 8, 9, 1, 5 };
    int N = sizeof(arr) / sizeof(arr[0]);

    // Function call
    quickSort(arr, 0, N - 1);
    cout << "Sorted array: " << endl;
    for (int i = 0; i < N; i++)
        cout << arr[i] << " ";
    return 0;
}
```

Output

```
Sorted array:

1 5 7 8 9 10
```

7.2.6 Complexity Analysis of Quick Sort:

Time Complexity:

Best Case: $\omega(N * logN)$

Average Case: $\Theta(N * logN)$

Worst Case: $O(N^2)$

Auxiliary Space: O(1) as no extra space is used

7.2.7 Advantages of Quick Sort:

- It is a divide-and-conquer algorithm that makes it easier to solve problems.
- It is efficient on large data sets.
- It has a low overhead, as it only requires a small amount of memory to function.

7.2.8 Disadvantages of Quick Sort:

- It has a worst-case time complexity of $O(N^2)$, which occurs when the pivot is chosen poorly.
- It is not a good choice for small data sets.
- It is not a stable sort, meaning that if two elements have the same key, their relative order will not be preserved in the sorted output in case of quick sort, because here we are swapping elements according to the pivot's position (without considering their original positions).

Practice Problems:

Given a doubly linked list. Sort it using merge sort algorithm

```
List: 10->20->8-17->5->13->4
Sorted list: 4->5->8->10->13->17->20
```

Algorithm

1. If head is NULL or list contains only one element then return list

2. Create two lists by dividing original list into 2 parts

3. Sort first and second part of list

4. Merge both sorted list